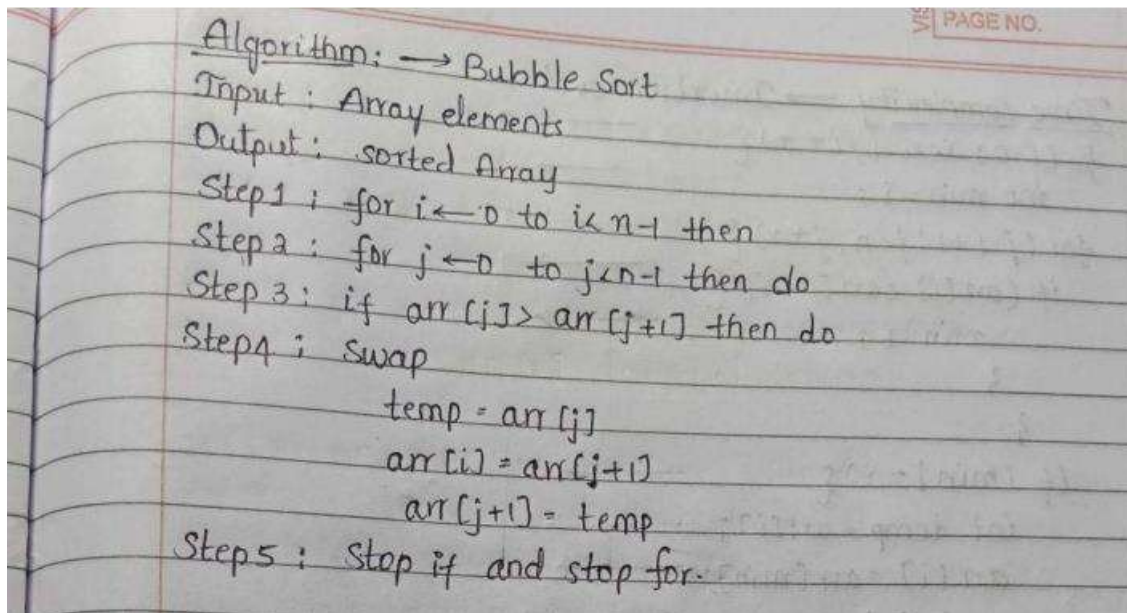


Practical –1

Aim: To Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort.

1)Bubble sort

Algorithm:



Program:

```
#include <iostream>

using namespace std;

int main() {
    int n;

    cout << "Enter the number of elements: ";

    cin >> n;

    int arr[n];

    cout << "Enter " << n << " elements: ";
```

```
for (int i = 0; i < n; i++) {  
    cin >> arr[i];  
}  
for (int i = 0; i < n - 1; i++) {  
    for (int j = 0; j < n - i - 1; j++) {  
        if (arr[j] > arr[j + 1]) {  
            int temp = arr[j];  
            arr[j] = arr[j + 1];  
            arr[j + 1] = temp;  
        }  
    }  
}  
cout << "Sorted array: ";  
for (int i = 0; i < n; i++) {  
    cout << arr[i] << " ";  
}  
return 0;  
}
```

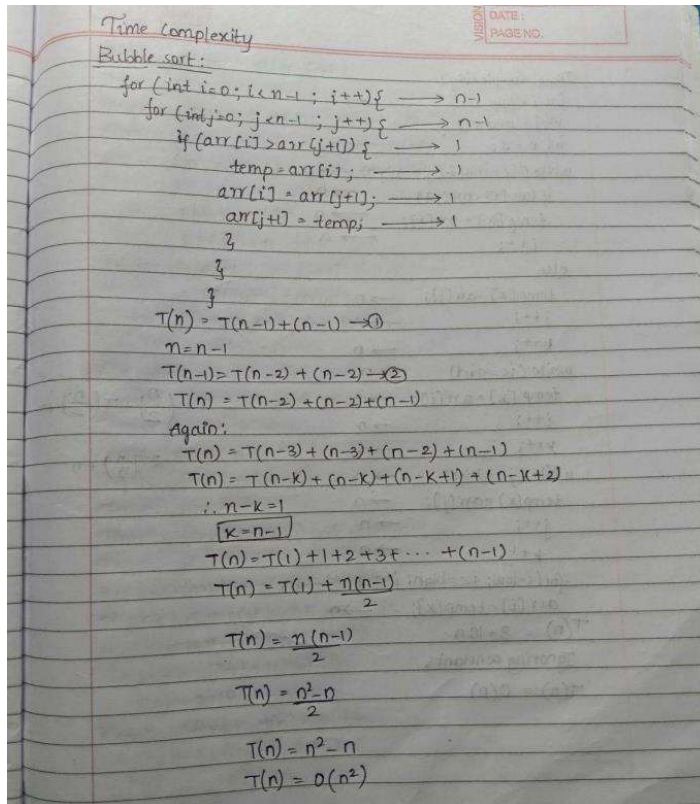
Output:

```
Enter the number of elements: 5  
Enter 5 elements: 9  
3  
7  
5  
1  
Sorted array: 1 3 5 7 9  
=== Code Execution Successful ===
```

Time complexity:

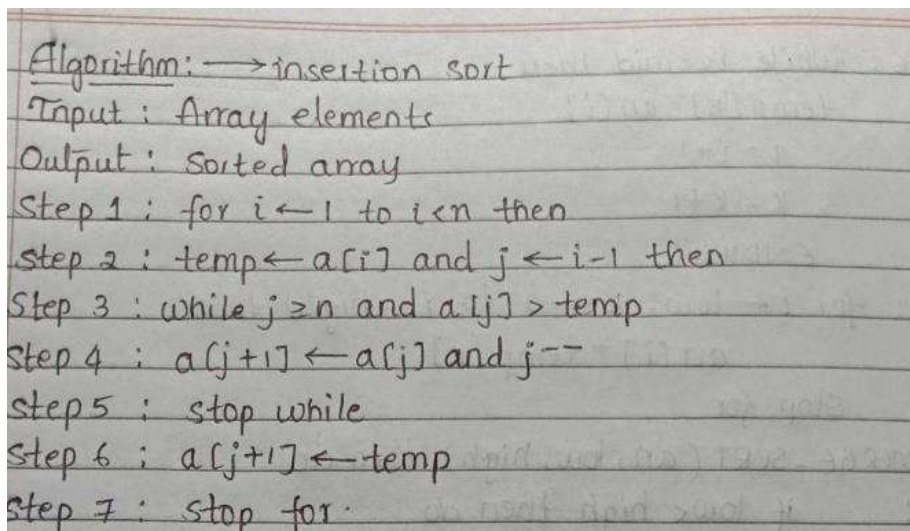
Name: V.Bhavya sri

Enrollment no: 92460118227



2) Selection Sort

Algorithm:



Program:

```
#include <iostream>
```

Name: V.Bhavya sri

Enrollment no: 92460118227

```
using namespace std;

int main() {
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
    int arr[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    for (int i = 0; i < n - 1; i++) {
        int key = arr[i];
        // Selection sort logic
    }
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    return 0;
}
```

Output:

```
Enter the number of elements: 5
Enter 5 elements: 1
4
3
6
9
Sorted array: 1 3 4 6 9

=== Code Execution Successful ===
```

Time complexity :

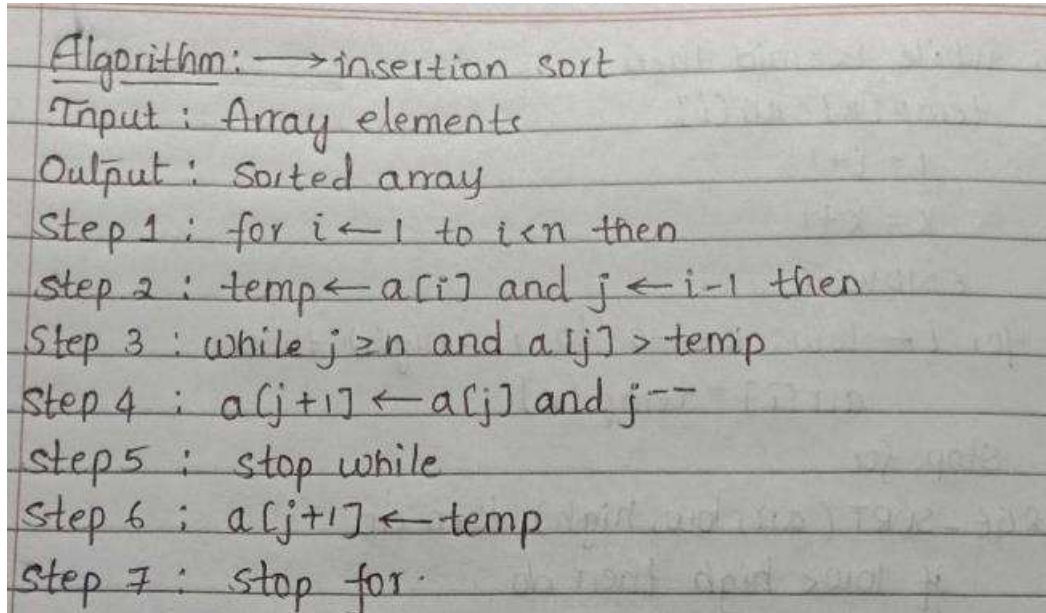
Time Complexity $\rightarrow$ Selection sort

```
for (i=0; i<n-1; i++) {  $\rightarrow n-1$ 
    int min=i;  $\rightarrow 1$ 
    for (j=i+1; j<n; j++) {  $\rightarrow n$ 
        if (a[j]<a[min])  $\rightarrow 1$ 
            min=j;  $\rightarrow 1$ 
        }
    if (min!=i)  $\rightarrow 1$ 
        swap(a[i], a[min]);  $\rightarrow 1$ 
    }
```

$$T(n) = \frac{n(n-1)}{2} = \frac{n^2-n}{2} = n^2-n \quad T(n) = O(n^2)$$

3) Insertion sort:

Algorithm:



Algorithm: $\rightarrow$ insertion sort
Input : Array elements
Output : Sorted array
Step 1 : for $i \leftarrow 1$ to $i \leftarrow n$ then
Step 2 : $temp \leftarrow a[i]$ and $j \leftarrow i-1$ then
Step 3 : while $j \geq 0$ and $a[j] > temp$
Step 4 : $a[j+1] \leftarrow a[j]$ and $j--$
Step 5 : stop while
Step 6 : $a[j+1] \leftarrow temp$
Step 7 : stop for

Program:

```
#include <iostream>

using namespace std;

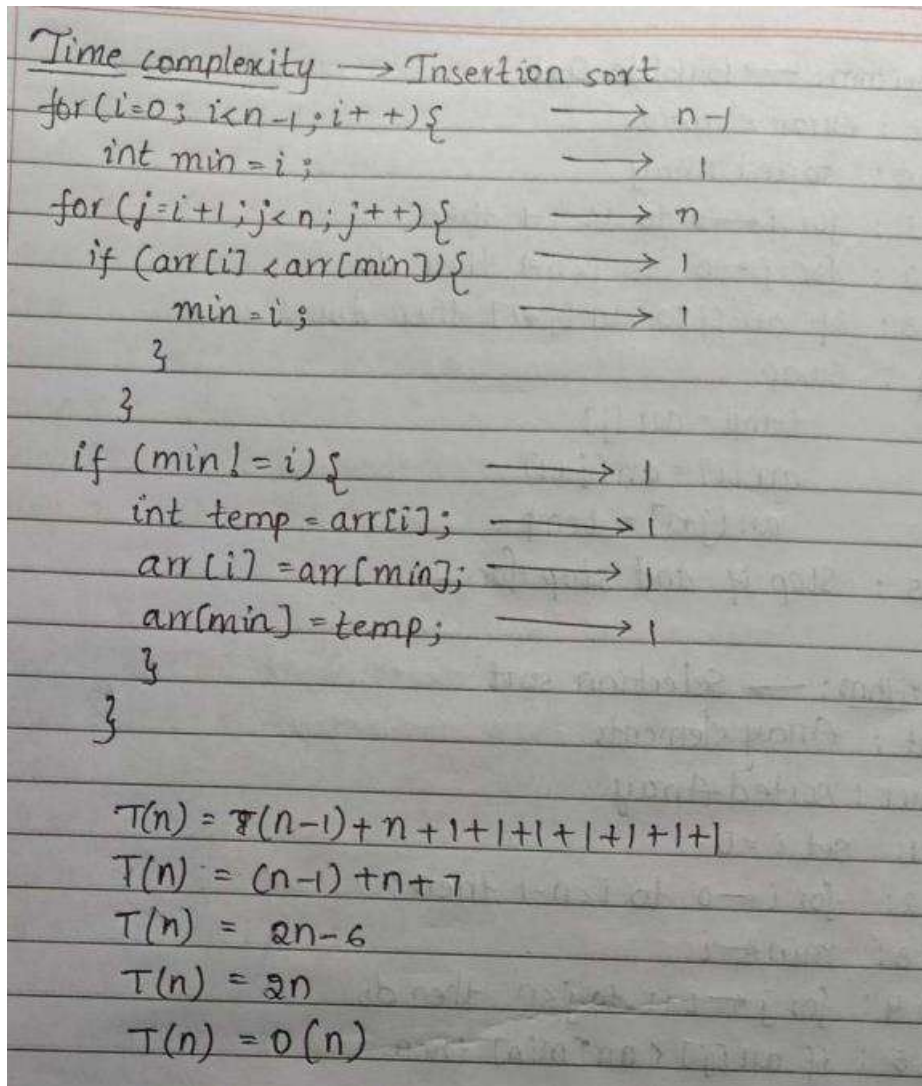
int main() {
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
    int arr[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    for (int i = 1; i < n; i++) {
```

```
int key = arr[i];  
int j = i - 1;  
while (j >= 0 && arr[j] > key) {  
    arr[j + 1] = arr[j];  
    j--;  
}  
arr[j + 1] = key;  
}  
cout << "Sorted array: ";  
for (int i = 0; i < n; i++) {  
    cout << arr[i] << " ";  
}  
return 0;  
}
```

Output:

```
Enter the number of elements: 5  
Enter 5 elements: 1  
4  
3  
6  
9  
Sorted array: 1 3 4 6 9  
=== Code Execution Successful ===
```


Time complexity:



4) Merge sort

Algorithm:

Algorithm: $\rightarrow$ Merge sort
Input: Array elements output: sorted array
Step 1: Merge (arr, low, mid, high) then
Step 2: Set $i = \text{low}$, $j = \text{mid} + 1$, $k = \text{low}$
Step 3: while $i \leq \text{mid}$ and $j \leq \text{high}$ then
Step 4: if $\text{arr}[i] \leq \text{arr}[j]$, then
 $\text{temp}[k] = \text{arr}[i]$
 $i = i + 1$
else
 $\text{temp}[k] = \text{arr}[j]$
 $j = j + 1$
END if
 $k = k + 1$
END while
Step 5: while $j \leq \text{high}$ then
 $\text{temp}[k] = \text{arr}[j]$
 $j = j + 1$
 $k = k + 1$
END while

Step 6: while $i \leq \text{mid}$ then
 $\text{temp}[k] = \text{arr}[i]$
 $i = i + 1$
 $k = k + 1$
END while
Step 7: for $i \leftarrow \text{low}$, $k \leftarrow 0$ to $i \leq \text{high}$ then
 $\text{arr}[i] = \text{temp}[k]$
stop for
Step 8: MERGE-SORT (arr, low, high) then do
Step 9: if $\text{low} < \text{high}$ then do
 $\text{mid} = (\text{low} + \text{high}) / 2$
 merge sort (arr, low, mid)
 merge sort (arr, mid + 1, high)
 merge (arr, low, mid, high)
Step 10: end if

Program:

```
#include <iostream>
```

```
using namespace std;
```

Name: V.Bhavaya sri

Enrollment no: 92460118227

```
void merge(int arr[], int low, int mid, int high) {  
    int i = low;  
    int j = mid + 1;  
    int k = 0;  
    int temp[high - low + 1];  
    while (i <= mid && j <= high) {  
        if (arr[i] < arr[j]) {  
            temp[k] = arr[i];  
            i++;  
        }  
        else {  
            temp[k] = arr[j];  
            j++;  
        }  
        k++;  
    }  
    while (i <= mid) {  
        temp[k] = arr[i];  
        i++;  
        k++;  
    }  
    while (j <= high) {  
        temp[k] = arr[j];  
        j++;  
        k++;  
    }  
}
```

```
        for (int i = low, k = 0; i <= high; i++, k++) {  
            arr[i] = temp[k];  
        }  
    }  
}  
  
void mergeSort(int arr[], int low, int high) {  
    if (low < high) {  
        int mid = (low + high) / 2;  
        mergeSort(arr, low, mid);  
        mergeSort(arr, mid + 1, high);  
        merge(arr, low, mid, high);  
    }  
}  
  
int main() {  
    int n;  
    cout << "Enter number of elements: ";  
    cin >> n;  
    int arr[n];  
    cout << "Enter " << n << " elements: ";  
    for (int i = 0; i < n; i++) {  
        cin >> arr[i];  
    }  
  
    mergeSort(arr, 0, n - 1);  
    cout << "Sorted array: ";  
    for (int i = 0; i < n; i++) {  
        cout << arr[i] << " ";  
    }
```

```
}
return 0;
}
```

Output:

```
Enter number of elements: 6
Enter 6 elements: 5
7
2
1
4
5
Sorted array: 1 2 4 5 5 7

=== Code Execution Successful ===
```

Time complexity:

Time Complexity - Merge Sort

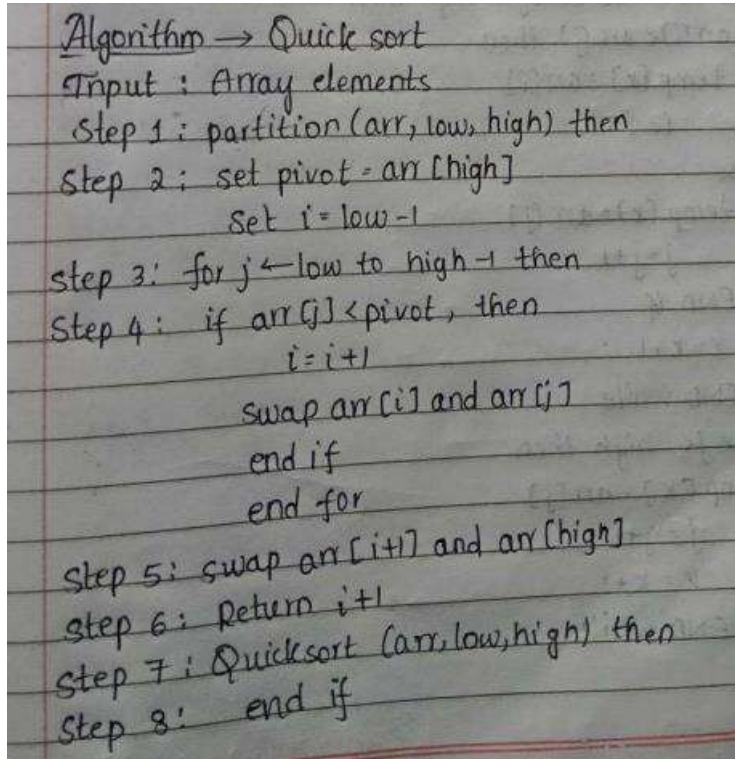
mergesort(arr, low, mid); $\rightarrow n/2$
mergesort(arr, mid+1, high); $\rightarrow n/2$
merge(arr, low, mid, high); $\rightarrow n$

$T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{2}\right) + n$
 $T(n) = 2T\left(\frac{n}{2}\right) + n$

Master Method:
 $T(n) = aT\left(\frac{n}{b}\right) + f(n)$
 $a=2, b=2, f(n)=n, p=0, k=1$
 $n^{\log_b a} = n^{\log_2 2} = n$
 $p(n) = n$
 $n^{\log_2 2} = n \therefore \log_2 2 = k$
case 2: $p > -1$
 $T(n) = O(n \log^{p+1} n)$
 $T(n) = O(n \log n)$

5) Quick sort

Algorithm:



Algorithm $\rightarrow$ Quick sort
Input : Array elements
Step 1 : partition (arr, low, high) then
Step 2 : set pivot = arr [high]
Set $i = \text{low} - 1$
Step 3 : for $j \leftarrow \text{low}$ to high-1 then
Step 4 : if arr [j] < pivot, then
 $i = i + 1$
swap arr [i] and arr [j]
end if
end for
Step 5 : swap arr [i+1] and arr [high]
Step 6 : Return $i + 1$
Step 7 : Quicksort (arr, low, high) then
Step 8 : end if

Program:

```
#include <iostream>
```

```
using namespace std;
```

```
int partition(int arr[], int low, int high) {
```

```
    int pivot = arr[high];
```

```
    int i = low - 1;
```

```
    for (int j = low; j < high; j++) {
```

Name: V.Bhavya sri

Enrollment no: 92460118227

```
if (arr[j] < pivot) {  
  
    i++;  
  
    int temp = arr[i];  
    arr[i] = arr[j];  
    arr[j] = temp;  
}  
}  
  
int temp = arr[i + 1];  
arr[i + 1] = arr[high];  
arr[high] = temp;  
  
return i + 1;  
}  
  
void quickSort(int arr[], int low, int high) {  
  
    if (low < high) {  
  
        int pi = partition(arr, low, high);  
  
        quickSort(arr, low, pi - 1);  
  
        quickSort(arr, pi + 1, high);  
    }  
}
```



```
}  
  
}  
  
int main() {  
  
    int n;  
  
    cout << "Enter number of elements: ";  
    cin >> n;  
  
    int arr[n];  
  
    cout << "Enter " << n << " elements: ";  
  
    for (int i = 0; i < n; i++) {  
        cin >> arr[i];  
    }  
  
    quickSort(arr, 0, n - 1);  
  
    cout << "Sorted array: ";  
  
    for (int i = 0; i < n; i++) {  
        cout << arr[i] << " ";  
    }  
}
```

```
return 0;
}
```

Output:

```
Enter number of elements: 5
Enter 5 elements: 9
2
6
1
6
Sorted array: 1 2 6 6 9

=== Code Execution Successful ===
```

Time complexity:

Time complexity $\rightarrow$ Quick sort

quicksort(arr, low, pi-1); $\rightarrow n/2$
 quicksort(arr, pi+1, high); $\rightarrow n/2$
 partition(arr, low, high); $\rightarrow n$

$$T(n) = T(n/2) + T(n/2) + n$$

$$T(n) = 2T(n/2) + n$$

Master method: $T(n) = 2T(n/2) + n$
 $a=2, b=2, f(n)=n, p=0, k=1$
 $n \log_b^a = n \log_2^2 = n$
 $\therefore \log_b^a = k$
 Case 2: $p > -1$
 $T(n) = \Theta(n \log^{p+1} n)$
 $T(n) = \Theta(n \log n)$

Conclusion:

The Bubble Sort algorithm successfully sorts the given elements in ascending order by repeatedly comparing adjacent elements and swapping them when they are in the wrong order. The algorithm is simple and easy to implement, but it is not efficient for large datasets. Its best-case time complexity is $O(n)$ when the array is already sorted, while the average-case and worst-case time complexities are $O(n^2)$. Therefore, Bubble Sort is mainly suitable for small datasets and educational purposes.